THE EXERCISE

Medication Safety Signal Monitor

Build a working product in Palantir Foundry that tells a hospital pharmacy safety officer which drugs are showing adverse events that are not on the label, and which have active recalls.

PLATFORM	TIME	DATA	ENDS WITH
Palantir Foundry + AIP	Three weeks from kickoff, part-time	Public sources only. No PHI.	A live demo and a one-page architecture note

01 The scenario

You are the engineer deployed to a health system's pharmacy safety team. The safety officer reviews a formulary of a few hundred drugs. Today she learns about emerging safety problems late, from FDA emails and word of mouth.

She wants one place that answers three questions each morning: **which drugs have adverse events being reported that the label does not mention**, **which drugs have an active recall**, and **what has changed since yesterday**. She also needs to trust it, so every flag must trace back to its source.

> **Why this problem.** Nothing here arrives clean. The sources disagree on drug names, update on different schedules, and mix structured records with documents. The interesting decisions are in how you join them, what you rebuild and when, and where you put the expensive steps. Those decisions are what this exercise is about.

02 What to build

A Pipelines

Ingest at least four sources, including one unstructured. Take them from raw to clean to an ontology with objects such as Drug, Label, Adverse Event, Recall and Signal. At least one pipeline must run incrementally.

B An AI function

Use an LLM to extract the labeled adverse reactions from drug label documents. Compare them with what is reported in FAERS to flag reactions that are frequent but unlabeled.

C An application

A Workshop app (or equivalent) with a ranked signal list, a drug drill-down showing the source label passage beside the extraction, recall alerts, and an action to triage a signal that writes back to the ontology.

D Monitoring STRETCH

Pipeline health: freshness, row-count anomalies, schema checks, build times. AI health: a hallucination check (is each extracted reaction in the source text?) scored on a small gold set you label, and drift in extraction output across label versions or model changes.

Keep the scope sane. Pick one therapeutic class or a formulary of 50 to 200 drugs. A narrow build that works end to end beats a broad one that half works.

03 Data sources

All are public and need no credentialing. Start with the core set. Adding your own sources is encouraged, and we want to hear why you chose them.

SOURCE	FORMAT	WHAT IT GIVES YOU	WHERE
FDA Adverse Event Reporting System (FAERS)	JSON API; quarterly bulk files	Millions of adverse event reports by drug and reaction. The high-volume source.	open.fda.gov/apis/drug/event open.fda.gov/data/downloads
DailyMed drug labels	PDF, SPL XML, label images	The official label for every marketed drug, with version history. Your unstructured source and the input to the AI function.	dailymed.nlm.nih.gov/dailymed/spl-resources-all-drug-labels.cfm
FDA drug enforcement reports	JSON API, updated weekly	Recalls, with classification, reason and affected product.	open.fda.gov/apis/drug/enforcement
RxNorm	REST API; bulk files	Standard identifiers linking brand names, generics and ingredients.	lhncbc.nlm.nih.gov/RxNav/APIs/RxNormAPIs.html
ClinicalTrials.gov	JSON API (v2)	Adverse events reported in trial results. A second opinion.	clinicaltrials.gov/data-api/api
IDEAS FOR EXTENDING IT			
PubMed	XML via E-utilities API	Case reports and safety studies, as supporting evidence.	ncbi.nlm.nih.gov/books/NBK25501
Medicare Part D spending by drug	CSV; API	Prescription volume, to weight signals by how widely a drug is used.	data.cms.gov
FDA Drug Safety Communications	Web pages, PDF	Published FDA warnings, to check your signals against.	fda.gov/drugs/drug-safety-and-availability/drug-safety-communications

A caution: these datasets are messier than their documentation suggests. Noticing where, and deciding what to do about it, is part of the exercise.

04 What to hand in

01 **The working build.** Pipelines, ontology, AI function and application, running in Foundry.

02 **A one-page architecture note.** A diagram of the data flow and ontology, plus the three or four decisions that mattered most and what you chose not to do.

03 **An optimization log.** Pick at least one pipeline, measure it, improve it, and show before and after: build time, compute, or cost of the LLM step. Say what you tried that did not help.

04 **A 20-minute demo,** followed by questions. Walk us through the app as the safety officer would use it, then go under the hood.

05 What we will look at

AREA	WHAT GOOD LOOKS LIKE
Architecture	A clear raw-to-ontology structure. Drug identity resolved once, in the right place. An ontology that a second engineer could extend without asking you how.
Optimization	Deliberate choices about incremental versus full rebuilds, partitioning, and where the LLM step sits so it is not re-run needlessly. Measured, not asserted.
Data judgment	You found the problems in the data, handled the ones that matter, and were honest about the ones you left.
The application	It answers the safety officer's three questions quickly, and every flag traces to its source. It proves the pipelines work.
AI reliability	You can say how often the extraction is wrong and how you know. Stretch: you would notice if it got worse.
Communication	The note and the demo explain trade-offs plainly, including what you would do with another month.

There is no single right design. We care more about why you made a choice than about which choice you made. A partial build with clear reasoning is a better result than a complete one you cannot explain.

06 Ground rules

01 **Environment.** We will sort out your Foundry access together at kickoff.

02 **Tools.** Use whatever you would use on the job, including AI coding assistants. You should be able to explain anything you submit.

03 **Data.** Public sources only. Do not use any patient-level or proprietary data.

04 **Questions.** Ask them. On a real engagement you would, and what you ask tells us something too.

05 **Assumptions.** Where the brief is silent, decide, and write the decision down.

06 **Scope.** If you are running out of time, cut breadth before depth, and tell us what you cut.

LIMITLESS INTELLIGENCE

Etter Hoang · Founder & Principal
ehoang@soragence.com · soragence.com